L3DML: Facilitating Geo-Distributed Machine Learning in Network Layer

Xindi Hou, Shuai Gao, *Member, IEEE*, Ningchun Liu, Fangtao Yao, Bo Lei, Hongke Zhang, *Fellow, IEEE*, and Sajal K. Das, *Fellow, IEEE*

***Abstract*—Geo-Distributed Machine Learning (GDML) aims to train large-scale machine learning models across geographically dispersed datacenters. However, the performance of GDML systems is constrained by the limited Wide Area Network (WAN) bandwidth and the presence of the straggler problem. Existing GDML designs often show contradictory effects in addressing these challenges, while in-network computing attempts are typically restricted to single datacenter environments rather than the more complex GDML scenarios. To overcome these limitations, this paper proposes L3DML to facilitate GDML using the P4-based Software-defined Network (SDN). Our approach incorporates three key innovations. Firstly, we introduce a novel network addressing scheme that enables location-specific in-network gradient aggregation for GDML, eliminating the need for parameter servers. Secondly, we utilize the P4 data plane to integrate lossless gradient transmission within switches. Thirdly, we address the straggler problem by employing a unique Deep Reinforcement Learning (DRL) model set and a corresponding rate synchronization routing approach. L3DML is implemented on a prototype system consisting of several Intel Tofino switches and the Spirent network emulator. Experimental results indicate that L3DML outperforms existing solutions in terms of goodput, model accuracy, and training speed gain for large-scale GDML.**

***Index Terms*—Distributed training, network-layer addressing, in-network computing, P4, deep reinforcement learning.**

I. Introduction

AI-ASSISTED applications leverage advanced AI models and large volumes of data to achieve enhanced performance. To accelerate model training, machine learning practitioners employ distributed training (DT), enabling training jobs to be distributed among multiple workers. This significantly reduces the training time to just a few hours or days [1]. However, the raw data is geographically dispersed and typically stored in nearby datacenters. For optimal performance, machine learning applications require a global view of all available raw data. This prompts the need to execute DT jobs across multiple datacenters, which is a concept known as Geo-Distributed Machine Learning (GDML) [2].

However, the performance of GDML system is limited by two major factors. The first factor involves **restrictions on the WAN resources** [3]: DT requires periodical gradient aggregation. These gradients indicate the direction for updating model parameters [4]. Each aggregation round involves numerous gradients, often ranging in size from hundreds of MB to GB [5]. Transmitting such large gradients over limited WAN bandwidth introduces additional latency to the GDML system. The second factor is the **straggler problem** [6]: The computational capabilities and raw data volume in different datacenters vary significantly. Datacenters that have completed their current training rounds need to wait for the slowest datacenter, known as the straggler, to perform gradient aggregation. This leads to a critical slowdown in the entire GDML system [7].

To address the above challenges, existing GDML solutions mainly focus on model synchronization and job scheduling. Model synchronization designs try to address WAN resource restrictions by eliminating insignificant gradient aggregation [3], [8], [9], [10], [11], [12]. However, these solutions exacerbate the straggler problem as the global models in different datacenters become inconsistent. Job scheduling designs seek to mitigate the straggler problem by periodically deploying DT jobs to different datacenters [13], [14], [15], [16], [17], [18], [19], [20], [21], [22], [23]. Yet, these approaches consume additional WAN resources during the deployment of DT jobs.

Since existing GDML solutions present conflicting outcomes, recent studies have attempted to address this problem using in-network computing [5], [24], [25], [26], [27], [28], [29], [30], [31]. These studies have found that modern programmable switches provide a flexible data plane, allowing gradients to be aggregated within a set of registers (aggregator) while being forwarded. These approaches effectively reduce the size of transmitted gradients in the network without introducing negative impacts. However, these approaches necessitate the use of parameter servers (PSs), which are

Received 28 April 2024; revised 22 September 2024 and 18 November 2024; accepted 23 November 2024. Date of publication 29 November 2024; date of current version 22 April 2025. This work is supported in part by the National Key Research and Development Project of China under Grant 2022YFB2901900, in part by the National Natural Science Foundation of China under Grants 62394324 and 61972026, and in part by the Beijing Natural Science Foundation under Grant 4242008. The work of S. K. Das is supported by the U.S. NSF grants under award nos. ECCS-2319995, OAC-2104078, and CNS-2030624. The associate editor coordinating the review of this article and approving it for publication was P. Papadimitriou. *(Corresponding author: Shuai Gao.)*

Xindi Hou, Shuai Gao, Fangtao Yao, and Hongke Zhang are with the School of Electronic and Information Engineering, Beijing Jiaotong University, Beijing 100044, China (e-mail: freezerburn@bjtu.edu.cn; shgao@bjtu.edu.cn; 21120153@bjtu.edu.cn; hkzhang@bjtu.edu.cn).

Ningchun Liu is with the Research Institute of Intelligent Networks, Zhejiang Lab, Hangzhou 311121, China (e-mail: nchliu@zhejianglab.edu.cn).

Bo Lei is with the Network Technology Research Department, Research Institute of China Telecom, Beijing 102209, China (e-mail: leibo@chinatelecom.cn).

Sajal K. Das is with the Department of Computer Science, Missouri University of Science and Technology, Rolla, MO 65409 USA (e-mail: sdas@mst.edu).

Digital Object Identifier 10.1109/TNSM.2024.3509031

centralized devices that perform gradient aggregation. This reliance on PSs presents challenges for cross-datacenter access. Furthermore, these approaches assume that all participating workers are connected via a shared top-of-rack (ToR) switch, assigning network-layer (L3) identifiers that are only addressable within a Local Area Network (LAN) for in-network aggregation. Consequently, the network-layer limitations of these approaches restrict their applicability to DT within a single datacenter, rather than the more complex GDML scenarios.

Therefore, this paper presents L3DML, an L3 design that employs an original aggregation task-aware addressing scheme and rate synchronization routing mechanism, to tackle the aforementioned challenges and facilitate GDML. Our approach encompasses three key innovations. Firstly, we propose a novel network addressing scheme that supports non-PS in-network aggregation for GDML. This scheme also enables location-specific aggregation, ensuring a balanced distribution of aggregation tasks among programmable switches. Secondly, we leverage the P4 data plane [32] to embed lossless gradient aggregation within switches, effectively reducing the gradient size while maintaining compatibility with existing DT protocols. Thirdly, we formulate the straggler problem in GDML scenarios and mitigate it by employing an original DRL model set and a corresponding rate synchronization routing approach. Our contributions are summarized as follows:

- We employ an Aggregation task IP address (AIP) to establish fine-grained identification for individual DT jobs. Then, an aggregation-aware AIP addressing scheme is proposed to support non-PS and location-specific in-network aggregation in GDML scenarios.
- The P4 language is utilized to develop data plane packet processing functions for AIP addressing, including AIP header processing and TCP header processing. These functions are embedded into the network to achieve lossless in-network gradient aggregation, both within individual datacenters and across multiple datacenters.
- We formulate the straggler problem in GDML scenarios by considering the dynamic WAN bandwidth and the computational capabilities of datacenters. Then, a decision transformer-based DRL rate synchronization routing approach is proposed to tackle the straggler problem.
- A prototype system is built using Intel Tofino switches and Spirent Attero-X network emulator to implement and evaluate L3DML. Experimental results demonstrate that L3DML outperforms existing solutions and brings notable performance enhancements for large-scale GDML.

The remainder of this paper is organized as follows. Section II discusses the related work. Section III presents motivations and an architectural overview of L3DML. Section IV details the design of L3DML. Section V presents the implementation and evaluation results. Section VI concludes the paper.

II. Related Work

In this section, we provide a brief overview of the related work to present a better understanding of our work.

A. Geo-Distributed Data Analytics

Geo-distributed data analytics faces challenges similar to GDML scenarios. Prior works aim to accelerate data analysis and fully utilize global computing resources in situations where data are globally generated [13], [14], [15], [16], [17], [18], [19], [20], [21], [23]. These studies present remarkable WAN bandwidth saving and performance improvement by applying job scheduling mechanisms among multiple datacenters. However, they mainly focus on computing jobs like map-reduce or SQL rather than sophisticated machine learning algorithms, making these solutions inapplicable for a distributed machine learning system.

B. Geo-Distributed Federated Learning

The major bottleneck for implementing federated learning across geographically dispersed clients is the communication overhead, as in the case of a GDML system. Previous research has focused on improving federated learning performance through approaches like client selection, routing algorithm design, and scheduling mechanisms [33], [34], [35], [36], [37]. However, these studies primarily address load balancing or security concerns, rather than specifically addressing the efficient operation of a GDML system in a WAN environment. Our approach differs from these studies because we consider the negative impact of WAN bandwidth limitations and the computational diversity among datacenters on the performance of the GDML system.

C. GDML System Designs

The GDML problem was initially discussed in [2] where the authors point out that a major challenge for running a machine learning system on geo-distributed data is the scarce WAN bandwidth. The authors also demonstrate that leveraging a proper algorithm to balance local computation with communication can greatly improve the performance of a GDML system. However, this solution requires modifications to machine learning algorithms, making it costly and algorithm-specific. Several studies have observed that not every round of gradient aggregation is necessary during the model training process, as most model updates only bring about minimal changes to the global model [38]. Based on this finding, various solutions have been proposed to eliminate unnecessary cross-datacenter communication and maintain an approximately accurate duplicate of the global model in each datacenter [3], [8], [11], [12], [39]. While these solutions effectively reduce communication overhead over the WAN, they come at the cost of sacrificing model accuracy.

Another challenge that a GDML system has to address is the straggler problem. To solve this, existing approaches employ job scheduling methods. These methods deploy DT jobs periodically to different datacenters, ensuring concurrent completion of all training jobs [22], [40], [41], [42], [43]. For instance, Liu et al. [22] propose a joint control scheme and re-configurable optical layers. They use a deterministic rounding algorithm for intra-job scheduling that dynamically allocates path and rate for each flow. For inter-job

scheduling, they design a delayed shortest weighted remaining time algorithm to schedule multiple jobs based on priorities. Bao et al. [40], [41] propose an online algorithm to schedule the arriving jobs in a shared cluster. By deciding the adjusted numbers of concurrent workers and PSs for each job, the computing resources of each datacenter are maximally utilized and model training can be completed in time.

Nevertheless, applying job scheduling in GDML scenarios may bring performance decrease, as periodically deploying training jobs to different datacenters will inevitably introduce additional WAN communication overhead.

D. In-Network Computing

The idea of aggregating data in the network does not originate from this paper. Prior works [44], [45] that target big data applications have shown that performing application-specific data aggregation at switch-attached middleboxes will bring notable performance benefits. DAIET [28] provides a proof-of-concept prototype for map-reduce applications running on a P4 emulator. Sanvito et al. [24] explore the feasibility of using the P4 data plane as the AI training accelerator, and demonstrate great performance advantages if the neural network to be trained is properly split. Liu et al. introduce NetReduce [29], an RDMA-compatible in-network data reduction architecture, to accelerate distributed deep neural network training. NetReduce is developed using FPGA and exhibits better scalability than the existing ring all-reduce architecture [46].

Since the current programmable switches offer a quite flexible data plane to customize packet processing, several attempts are made on dedicated hardware switches to implement in-network computing. For instance, iSwitch [27] is an in-switch acceleration solution for distributed reinforcement learning. SwitchML [5], ATP [25], and SHARP [26] focus on protocol designs that can natively support gradient aggregation for distributed machine learning applications in the Intel Tofino switches. A2TP [30] and SAINA [31] focus on addressing the impact of slower workers on the overall performance of distributed training by considering the aggregation order.

However, all these studies necessitate central PSs and assume that in-network computing occurs exclusively within a single datacenter, specifically, in a rack-scale structure. These studies primarily address the performance obstacles of in-network computing, rather than efficiently implementing a GDML system over the WAN. Moreover, specific protocols are necessary to handle packet loss, making it incompatible with the existing network and DT applications.

E. Rate Synchronization Routing

Distributed machine learning has the bottleneck effect during the gradient aggregation process, where the time cost for a round of gradient aggregation is significantly impacted by the slowest worker. To address this issue, some recent approaches regarding rate synchronization routing have been proposed, enabling gradients from multiple workers to reach their aggregation destination in an appropriate order. Chen et al. [47], [48] respectively formulate the rate synchronization problem in wired and wireless networks for distributed training. They also propose a data aggregation-aware routing algorithm to accelerate gradient aggregation, but most of their efforts are theoretical. GRID [7] is a gradient-based routing design that natively supports in-network aggregation in the context of LAN networks, aiming to ensure that the gradients from different workers synchronously arrive at the switch for aggregation. GRID is implemented in a small-scale testbed based on the Intel Tofino switches and can achieve high training throughput.

Our work differs from the above approaches in two aspects. First, they only consider a wide variety of workers to conduct rate synchronization irrespective of the computational capacities of various datacenters and WAN bandwidth constraints. Second, compared to the straggler problem in a single datacenter, solving the straggler problem in GDML scenarios involves a more complex parallel per-hop decision-making process to establish an optimal aggregation order. Therefore, their designs are only suitable for the DT within a single datacenter. In contrast, our work analyzes the complex cross-datacenter gradient aggregation process and considers both the computing resources of different datacenters and the WAN resources, making our designs applicable to GDML scenarios.

III. Motivations and Architectural Overview

A. Our Motivations

Our goal is to facilitate GDML by: 1) reducing the size of gradients transmitted over the WAN to overcome WAN bandwidth constraints, and 2) mitigating the straggler problem so that the training performance is not bottlenecked by the slowest datacenter. To this end, L3DML aims to tackle the limitations present in current in-network computing approaches and rate synchronization routing designs. However, addressing the aforementioned limitations in the network layer is challenging, as it necessitates a GDML system that meets the following requirements:

1) Non-PS aggregation: Most of the DT applications are based on the PS-worker architecture [49], where the gradients from multiple workers are aggregated within one or multiple PSs. However, in GDML scenarios, it is difficult to access PSs across datacenters due to the utilization of Network Address Translation (NAT) technology. Another DT architecture, the all-reduce [46], has the same issue.

2) Inter-rack aggregation: Programmable switches are commonly used as ToR switches for near-server computation [50]. Nevertheless, the participating workers are typically distributed across various racks in a datacenter. Thus, it is necessary to expand the in-network aggregation capability beyond a rack-scale network environment.

3) Processing transport layer: Existing DT applications rely on Transmission Control Protocol (TCP) to handle packet loss, which exceeds the capabilities of network switches. To enable lossless gradient transmission without modifying protocols, the data plane needs to possess transport layer processing logic.

4) Cross-datacenter aggregation: In GDML scenarios, gradient aggregation is needed for all participating workers

across datacenters. To achieve the most accurate model, the gradients from each worker cannot be discarded. Given that programmable switches are commonly deployed within datacenters, the data plane should be capable of performing Cross-datacenter aggregation.

5) Parallel and collaborative routing: Making rate synchronization routing decisions requires considering the disparities in computational capabilities and dynamic changes in bandwidth between datacenters. Additionally, parallel and collaborative distribution of routing decisions is needed for participating datacenters.

6) Deployability and compatibility: Deploying dedicated hardware in the WAN can be expensive. Therefore, a GDML system must support deployment using the existing network infrastructure, protocol stacks, and sockets. Moreover, the proposed mechanisms should be applicable to various DT applications without altering the existing DT framework.

In response to the above requirements, we are motivated to propose a novel L3 design, referred to as L3DML.

P4 is a high-level programming language. It allows network operators to customize the data plane behavior for packet processing in a flexible and protocol-independent manner [32]. L3DML leverages the P4-based SDN and incorporates innovative network-layer addressing and in-network computing techniques to overcome WAN bandwidth constraints. Reinforcement learning can adapt to changing environments and excel in problems that involve a sequence of decisions. This approach is particularly effective at learning strategies that prioritize long-term rewards over immediate gains in dynamic and complex optimization problems. Therefore, L3DML introduces the DRL method to address the straggler problem.

The objective of our research is to delve into the development of GDML systems and offer practical insights for their implementation and deployment.

B. Architecture of L3DML

In L3DML, we focus on a scenario where multiple workers in different datacenters run the same DT job to collaboratively train a global model. The architecture of L3DML is presented in Fig. 1, which comprises the intra-datacenter (I-DC) gradient aggregation, cross-datacenter (X-DC) gradient aggregation, and control plane functions.

I-DC aggregation: In the I-DC aggregation, L3DML aggregates gradients within a single datacenter into a single set before conducting X-DC aggregation. In the datacenter network, a group of workers are connected through a ToR switch, whereas multiple racks are interconnected via an aggregation network in cases where participating workers belong to different racks. Both the ToR switches and aggregation network are programmable. Given the difficulty of workers accessing PSs across datacenters, L3DML proposes the AIP addressing scheme. By employing a virtual PS address, workers can use the PS-worker architecture without deploying PSs within datacenters, enabling non-PS gradient aggregation. Additionally, L3DML introduces a lossless in-network gradient aggregation mechanism based on a series of TCP header processing logic. The data plane can steer gradient packets to specific aggregation points based on their AIP.

X-DC aggregation: To obtain the final aggregated results, the X-DC aggregation process is performed when the I-DC aggregation process is completed. To maximize the utilization of WAN bandwidth across different datacenters during X-DC aggregation, we utilize the all-reduce architecture, where the gradients from faster datacenters are sequentially aggregated with those from slower datacenters hop-by-hop. The gradients eventually reach the straggler for the final aggregation. Subsequently, the final aggregated results are multi-cast to all participating datacenters for model update. To address the straggler problem during the X-DC aggregation process, L3DML utilizes a decision transformer-based DRL rate synchronization routing approach, which considers the dynamic WAN bandwidth and computational capabilities of datacenters to determine the optimal aggregation order among datacenters.

Control plane functions: L3DML utilizes the SDN architecture and deploys three main functions for control plane management. The *I-DC Aggregation Scheduler* manages the DT job allocation within datacenters and determines the data plane behavior according to specific DT jobs in the I-DC aggregation. The *Flow Table Manager* distributes flow tables to on-path switches to implement in-network gradient aggregation and packet delivery. The *X-DC Aggregation Scheduler* maintains central knowledge and makes routing decisions among datacenters using the DRL method to facilitate the X-DC aggregation. Multiple *X-DC Aggregation Schedulers* achieve consistency through periodic notifications in the context of a cluster.

IV. L3DML Designs

A. AIP Addressing

Existing GDML system designs remove insignificant gradient aggregation to reduce the gradients transmitted over the WAN. L3DML performs gradient aggregation in every round to maintain an accurate global model and employs in-network aggregation to minimize gradient size. Contrary to state-of-the-art in-network aggregation solutions that require PS deployment and restrict aggregation to ToR switches, L3DML utilizes a novel AIP addressing scheme for non-PS and location-specific aggregation. It steers gradient packets from participating workers to designated switches for aggregation using a virtual address. In this section, we outline the AIP design and the AIP addressing workflow.

1) AIP Design: Current DT applications need to specify the IP address of the PS for gradient aggregation. However, cross-datacenter PS access can be challenging. To overcome this, L3DML adopts AIP as a virtual PS address to enable non-PS gradient aggregation. AIP represents a specific round of gradient aggregation for a given DT job and is addressable for any datacenter, as participating workers may be physically located in different datacenters. The format of AIP is depicted in Fig. 2. The AIP possesses the following functions:

Compatible with IP network: Using existing protocol stacks or sockets allows workers to construct and send AIP packets. This approach avoids additional deployment overhead

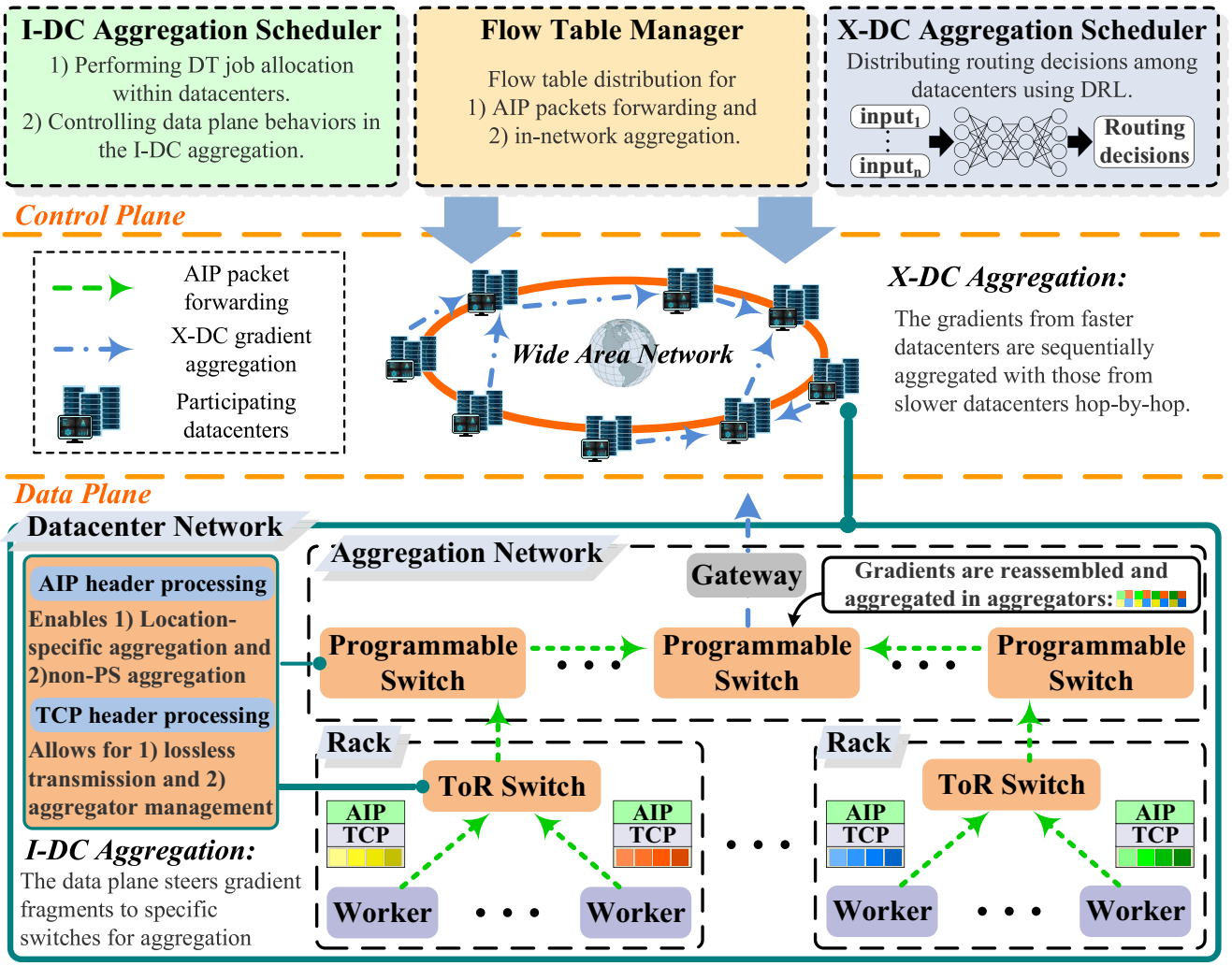


Fig. 1. Architecture of L3DML.

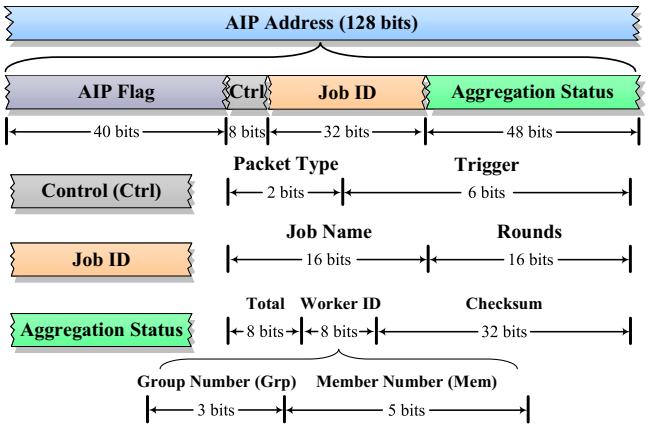


Fig. 2. AIP format.

from protocol modifications. Therefore, AIP is implemented using the IPv6 namespace to ensure compatibility with the IP protocol. The fields for AIP addressing occupy a total of 88 bits in the IPv6 namespace. To avoid IPv6 namespace wastage, 40 bits of *AIP Flag* are included as a data plane trigger for the AIP packet processing pipeline. In L3DML, the value of the *AIP Flag* is set to '0xff02ab0001'. Packets with AIP Flag values other than '0xff02ab0001' will be processed as regular IPv6 packets. To facilitate deployment, AIP addressing is specifically implemented within each datacenter.

Data plane behavior control: In AIP, the *Ctrl* field, occupying 8 bits, is used to control the packet processing behavior in the data plane. This includes forwarding the packet and conducting gradient aggregation. The *Ctrl* field consists of the *Packet Type* field and the *Trigger* field. The *Packet Type* field, with a size of 2 bits, specifies the type of the packet. A value of 01, 10, or 11 in the *Packet Type* field respectively indicates an I-DC aggregation request, X-DC aggregation request, or aggregation result packet. The *Trigger* is a 6-bit number used to specify the location where the gradients are aggregated. Whenever the packet is forwarded, its *Trigger* value decreases by 1. Upon reaching 0, the data plane executes in-network gradient aggregation.

Fine-grained identification for DT jobs: AIP utilizes the *Job ID* field to enable fine-grained identification of DT jobs. To accommodate the maximum number of indexes supported by the aggregator of the P4 data plane (2^{32}), the *Job ID* field

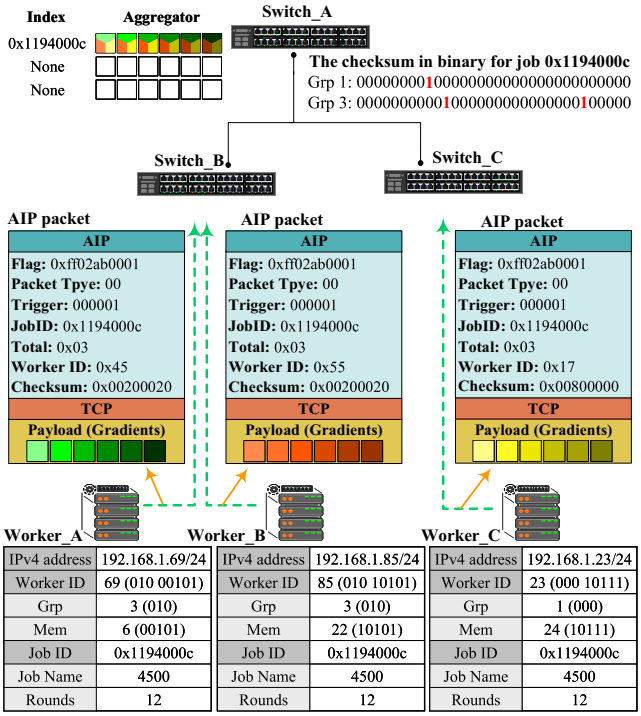
is set to 32 bits. This ensures that gradients generated by workers executing the same DT job are aggregated within the same aggregator of a programmable switch. The *Job ID* field consists of the *Job Name* field and the *Rounds* field. The *Job Name* is allocated 16 bits and serves as a unique identifier for each individual DT job, often represented by the TCP port number used by that job. The *Rounds* field occupies 16 bits and indicates the current round number of the aggregation process. Hence, each round of gradient aggregation in a specific DT job is allocated with a unique *Job ID*.

Aggregation verification: AIP leverages the 48-bit *Aggregation Status* field to determine whether the I-DC aggregation process is finished. The *Aggregation Status* consists of the *Total*, the *Worker ID*, and *Checksum* field. The *Total* field takes up 8 bits and is used to indicate the total number of participating workers in a datacenter for the specific *Job ID*, with a maximum support of 256 workers. It is also utilized to partially determine the completion of I-DC aggregation. Each worker in a datacenter is assigned a unique 8-bit *Worker ID*, which could be the host number of its IPv4 address since the workers are interconnected through the Local Area Network (LAN). To simplify the verification process of aggregation completion, workers are divided into 8 groups via a 3-bit *Group Number* (Grp) field, with each group comprising 32 members that are identified by their respective *Member Number* (Mem). The *Checksum* field is a 32-bit one-hot code utilized for verifying the completion of I-DC aggregation for a specific *Job ID*. For the given *Group Number*, each bit in the *Checksum* field indicates the participation of an individual member during the aggregation. The detailed verification process is described in Section IV-A2.

The proposed AIP addressing scheme offers several advantages. Firstly, it enhances compatibility with traditional IP networks without requiring modifications to protocol stacks or sockets. Secondly, it enables location-specific gradient aggregation, expanding aggregation beyond ToR switches. This flexibility allows the control plane to select appropriate aggregation points, distributing the aggregation workload across multiple switch aggregators, and maximizing switch bandwidth usage. Thirdly, the AIP serves as the virtual PS address for DT jobs. It is included as the destination address in gradient packets. Programmable switches utilize the AIP fields to process these packets, enabling non-PS in-network aggregation without altering existing DT architectures. Furthermore, since AIP is addressable for participating datacenters, AIP packets can be routed over the WAN using segment routing (SR), facilitating X-DC aggregation in GDML scenarios.

2) AIP Addressing Workflow: For a comprehensive understanding of the AIP addressing, we divide the I-DC aggregation process into three phases: **worker sending gradients**, **gradients delivering and aggregation**, and **aggregation verification**. Additionally, an example is provided to illustrate the AIP addressing workflow, as shown in Fig. 3.

Worker sending gradients: Consider a scenario where a datacenter comprises three workers performing the DT job with a *Job ID* of '0x1194000c' (TCP port 4500, round 12). The *Worker IDs* of these individuals correspond to the host



IPv4 address	192.168.1.69/24
Worker ID	69 (010 00101)
Grp	3 (010)
Mem	6 (00101)
Job ID	0x1194000c
Job Name	4500
Rounds	12

IPv4 address	192.168.1.85/24
Worker ID	85 (010 10101)
Grp	3 (010)
Mem	22 (10101)
Job ID	0x1194000c
Job Name	4500
Rounds	12

IPv4 address	192.168.1.23/24
Worker ID	23 (000 10111)
Grp	1 (000)
Mem	24 (10111)
Job ID	0x1194000c
Job Name	4500
Rounds	12

Fig. 3. The AIP addressing workflow.

numbers of their IPv4 addresses. These workers are categorized into two distinct groups based on their respective *Worker IDs*. The value of the *Checksum* field in the AIP packet is determined by the participation of workers. As an example, in group 3, the gradients of members 6 and 22 require aggregation. Thus, the values of the 6th and 22nd bits in the *Checksum* of group 3 are set to 1. Notably, AIP packets sent by workers of the same group carry the *Checksum* value associated with that specific group.

Gradients delivering and aggregation: Assuming switch_A is selected as the aggregation point by the control plane, the *Trigger* value in the AIP packet is set to 1. When the AIP packet sent by the worker reaches their respective ToR switches (switch_B and switch_C), the switches decrement the *Trigger* value by 1 and forward the packet to switch_A based on the flow table. Upon reaching switch_A, the AIP packet triggers an aggregation action due to the *Trigger* value being 0. Switch_A stores and aggregates the received gradients into the aggregator indexed as '0x1194000c', ensuring that all gradients of this DT job are aggregated in the same aggregator.

Aggregation verification: The programmable switch utilizes the *Total* and *Checksum* fields to verify the aggregation status of worker gradients. When the aggregation is finished, the values of the *Total* and the *Checksum* fields comply with Eq. (1), where the binary indicator $f_{i,j}$ is obtained by Eq. (2). The term $f_{i,j} * 2^i$ in this equation is referred to as the *Check Code* of member i in group j. In this example, switch_A employs local user-defined counters to record the received *Total* value and the *Checksum* value of each group. Using the received *Group Number* and *Member Number*, switch_A calculates and accumulates the *Check Code* of the individual worker in local counters until Eq. (1) is satisfied. Once the I-DC aggregation is completed, the corresponding aggregator

Example of the Ctrl Table:

Key={hdr.aip.ctrl}	Action
01-000101	NoAction
00-111111	drop()
00-000000	NoAction
...	...

Example of the Fwd Table:

Key={hdr.aip.jobid}	Action
0x1194000c	set_output_port(ens2)
0x450a001a	set_output_port(ens4)
0x926d8a9c	drop()
...	...

Fig. 4. Example flow tables of L3DML.

is released, and the gradients are transmitted to the WAN for the subsequent X-DC aggregation.

$$\begin{cases} \forall Grp : \exists Checksum_{Grp} = \sum_{i=0}^{31} (f_{i,Grp} * 2^i) \\ Total = \sum_{j=0}^{7} \sum_{i=0}^{31} f_{i,j} \end{cases} \tag{1}$$

$$f_{i,j} = \begin{cases} 1, & member\ i\ in\ group\ j\ is\ a\ participant \\ 0, & otherwise \end{cases} \tag{2}$$

B. P4-Based AIP Header Processing

AIP addressing differs significantly from traditional IP addressing, necessitating programmability in the data plane for customized packet processing. However, implementing AIP addressing using programmable switches presents challenges. Firstly, current in-network aggregation approaches often rely on packet recirculation, which leads to suboptimal line-rate performance when handling small gradient packets. Secondly, programmable switches are incapable of packet generation, preventing them from activating X-DC aggregation. Thirdly, aggregation verification requires stateful processing, resulting in complexity to the data plane. To address these issues, L3DML leverages the protocol-independent P4 language to design AIP header processing logic.

The primary tasks of the AIP header processing involve directing incoming AIP packets to the appropriate egress port and managing aggregation. These tasks are achieved by executing simple match-action operations using flow table entries obtained from the control plane *Flow Table Manager*, without relying on packet recirculation. L3DML introduces two types of flow tables for AIP header processing, as depicted in Fig. 4 (To make it easier for observation, '-' is inserted between different fields).

The *Ctrl Table* utilizes the *Ctrl* as the match field (key). It activates the control plane *Flow Table Manager* to distribute the necessary flow table entries for processing AIP packets. In cases where the *Trigger* value of an AIP packet is excessively large, the packet will be dropped to avoid network looping and aggregation failure. The *Fwd Table* leverages the *JobID* as the match field to determine the egress port of a given AIP packet. AIP packets with incorrect *JobID* will be dropped. When a *packet_in* message is received, the control plane *Flow Table Manager* will distribute the *Fwd Table* entries along with the *Ctrl Table* entries.

L3DML utilizes user-defined metadata and stateful counters to facilitate AIP header processing. Fig. 5 illustrates the P4 pipeline design in L3DML including the following functions.

Packet parsing and deparsing: The P4 *Parser* is used to extract the required fields and relevant metadata from the packet header according to the user-defined AIP header structure. The P4 *Deparser* is leveraged to reassemble necessary headers into the packet for output format declaration.

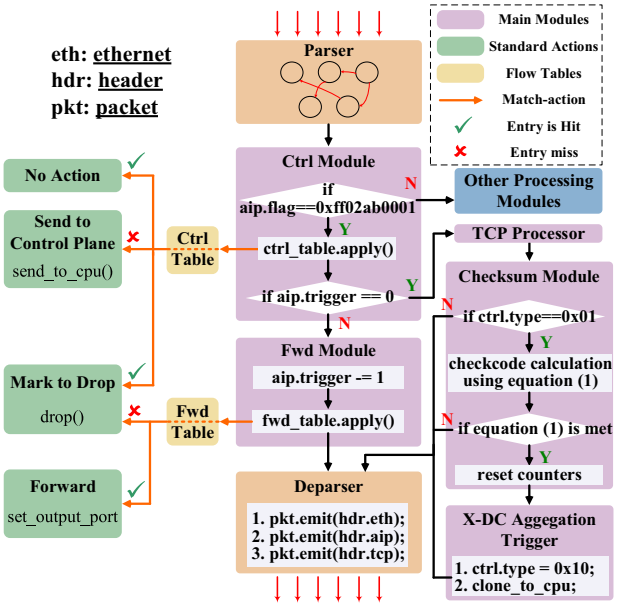


Fig. 5. The P4 pipeline design for AIP header processing.

AIP processing activation: L3DML employs the *Ctrl Module* to identify the packet type and determine the subsequent processing steps. Specifically, if the value of the *AIP Flag* is '0xff02ab0001', the AIP header processing will be activated. Otherwise, other processing logic, such as IPv6 packet processing, will be applied. Subsequently, the *Ctrl Table* is used for match-action operations to activate control-plane flow table distribution and filter out packets with excessively high *Trigger* values. Following this, the *Ctrl Module* determines whether to perform gradient aggregation by evaluating the *Trigger* value in the packet. If the *Trigger* value is 0, the packet is delivered to the *TCP Processor* (described in Section IV-C) for gradient aggregation. Otherwise, the packet is sent to the *Fwd Module*.

Gradient packet forwarding: L3DML designs the *Fwd Module* to manage gradient packet forwarding. It begins by subtracting 1 from the *Trigger* field in the received packet through metadata operations. Then, the match-action is executed using the *Fwd Table*, which determines the corresponding egress port based on the *JobID* field. In case of a table miss or an incorrect *JobID*, the packet will be dropped.

I-DC aggregation verification: To enable stateful processing across packets, L3DML introduces the *Checksum Module*. This module employs external methods to attach counters to the flow tables, triggering counting whenever an AIP packet completes a match-action. As a result, it enables the counting of participating workers and the accumulation of *Check Codes* for each group. The *Checksum Module* compares the counter values with the *Checksum* and *total* fields of the received AIP packet. Once condition (1) is satisfied, it verifies the completion of I-DC aggregation, and the corresponding counters will be reset. The verification is unnecessary for X-DC aggregation as it is performed hop-by-hop between datacenters.

X-DC aggregation activation: L3DML introduces the *X-DC Aggregation Trigger* to activate X-DC aggregation initially.

This module modifies the AIP packet to an X-DC aggregation request packet (type=0x10) and clones it to the control plane's *X-DC Aggregation Scheduler* for activation notification. *X-DC Aggregation Scheduler* periodically sends X-DC aggregation request packets (without payloads) to the switch pipeline, triggering the *TCP Processor* to send out gradient fragments from the aggregator and route them to the next-hop datacenter. The control plane can then utilize flow table entries to perform in-network aggregation and continue activating the next X-DC aggregation process using the same logic.

AIP header processing applies simple match-action operations and minimizes the utilization of complex processing logic. This strategy avoids packet recirculation and effectively mitigates the negative impact on data plane performance.

C. P4-Based TCP Header Processing

In GDML scenarios, the gradients produced by a worker in a training round often exceed the Maximum Transmission Unit (MTU) of the Ethernet. Consequently, the DT applications have to partition the gradients into multiple fragments. Hence, mechanisms are required to ensure lossless gradient transmission.

Existing in-network aggregation approaches typically require a dedicated protocol to enable lossless gradient transmission, such as the ATP protocol in [25] and the shadow copy mechanism in [5], rendering them incompatible with the sockets used by traditional DT applications. Moreover, traditional GDML schemes usually depend on the TCP connections between workers and PS to ensure lossless gradient transmission. However, L3DML utilizes non-PS aggregation, which may prevent the establishment of an end-to-end TCP connection. Therefore, L3DML utilizes the P4 language to integrate lossless gradient transmission into the data plane.

Achieving lossless gradient transmission in the P4 data plane is difficult since both ends of a TCP connection must possess local caches for caching and reassembly of gradient fragments. Yet, the P4 data plane only supports per-packet processing, lacking long-term packet caching and gradient fragment reassembly capabilities. Furthermore, it lacks TCP processing logic to ensure the sequential arrival of gradient fragments. To handle this, L3DML employs the *TCP Processor* to support gradient fragment reassembly and ensure complete gradient arrival.

The *TCP Processor* reassembles gradient fragments within P4 aggregators using the segment number in the TCP header. To achieve this, the TCP segment size must be smaller than the MTU, which can be accomplished by adjusting the local TCP configuration of the worker. Since gradient confirmation relies on ACK (Acknowledgement) packets, the *TCP Processor* performs ACK confirmation through field modification and packet forwarding. Fig. 6 illustrates the P4 pipeline design for the *TCP Processor*, achieving the following functions:

TCP connection establishment: We design the *Connection Manager* for establishing TCP connections between P4 switches and workers. It determines if an incoming packet is a TCP connection establishment request based on the *SYN* and *ACK* fields in the TCP header. A request is identified

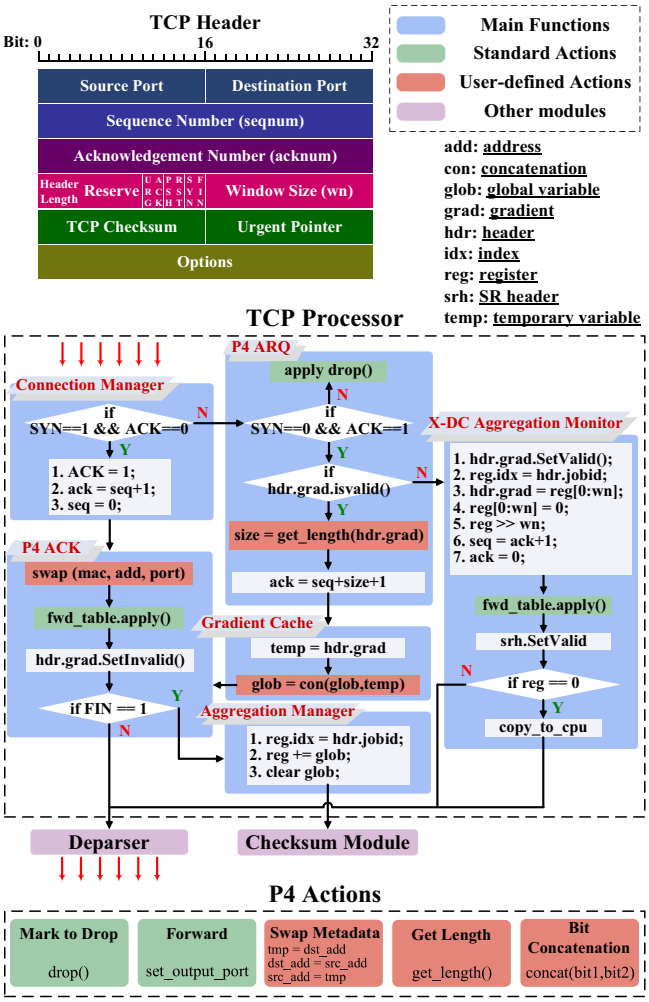


Fig. 6. The P4 pipeline design for the TCP processor.

when the *SYN* value is 1 and the *ACK* value is 0. Otherwise, the packet is sent to the *P4 ARQ* function. For establishment requests, the *Connection Manager* sets the *ACK* field to 1 and assigns *seqnum+1* as the value of the *acknum* field, confirming the starting sequence number of the gradient byte stream to the worker. As the P4 switch does not send gradients to the worker, the TCP connection is simplex. Hence, the *seqnum* field is set to 0, and the packet is delivered to the *P4 ACK* function. The TCP connection established by the *Connection Manager* is a pseudo connection, ensuring compatibility with existing DT applications by making them network-insensitive.

Enabling ARQ (Automatic Repeat reQuest): To enable the sequential and lossless arrival of gradient fragments, we design a per-packet ACK confirmation mechanism and employ the *P4 ARQ* function. It examines the *SYN* and *ACK* fields (*SYN* is 0 and *ACK* is 1) to identify gradient packets. Packets with incorrect *SYN* and *ACK* are dropped. For gradient packets, the *P4 ARQ* verifies the payload (gradients) validity. Invalid packets are flagged as X-DC aggregation requests and forwarded to the *X-DC Aggregation Monitor*, while valid packets trigger the user-defined action *Get Length* to determine the byte length of the gradients, which is then written to the *acknum* field and sent as an ACK message to the worker to confirm the sequence number of the received gradient bytes.

Finally, the packet is delivered to the *Gradient Cache* for gradient reassembly. If there are lost gradient fragments, the worker initiates packet re-transmission based on the received ACK message.

Gradient fragments reassembly: In the P4 data plane, the aggregator is an array. The length of gradient fragments is usually smaller than the length of the aggregator. However, segmented writing of gradients is not supported by P4 aggregators. To overcome this limitation, we introduce the *Gradient Cache*. It stores fragments in temporary variables and utilizes the non-standard P4 action *Bit Concatenation* to concatenate fragments from multiple packets using a global variable.

Enabling ACK confirmation: To facilitate ACK confirmation to the worker, we introduce the *P4 ACK* function. It utilizes the user-defined action *Swap Metadata* to exchange the source and destination addresses of the received packet. Then, the match-action is performed using the *Fwd Table* to determine the egress port of the packet, allowing the ACK confirmation to be sent back to the corresponding worker. Subsequently, the packet payload is marked as invalid since the TCP connection is simplex. If the *FIN* value is 1, indicating the completion of gradient fragment transmission, the packet is directed to the *Aggregation Manager*. Otherwise, the packet is delivered to the *Deparser*.

In-network aggregation: Upon concatenation of all gradient fragments, the *Aggregation Manager* is devised to aggregate the complete gradients into the specified aggregator indexed by *JobID*. Once the in-network aggregation is accomplished, the global variables are cleared, and packets are forwarded to the *Checksum Module*.

Maintaining X-DC aggregation: To maintain X-DC aggregation, we developed the *X-DC Aggregation Monitor* to handle X-DC aggregation requests. It uses bit-shifting operations to extract gradient segments (sized by *wn*) from the aggregator (indexed by *JobID*) and places them in the packet payload. Then, AIP packets are encapsulated with an SR header and sent to the *Deparser* for X-DC aggregation. The *X-DC Aggregation Monitor* fills the Segment ID (SID) of the next-hop datacenter, obtained from the routing algorithm in Section IV-D4, into the SID list of the SR header. This enables the AIP packet to be sent to the next-hop datacenter. As the AIP packet is globally addressable, the *X-DC Aggregation Monitor* of the next-hop datacenter will repeat the aforementioned AIP and TCP header processing process, ultimately completing the X-DC aggregation. Once the aggregator is empty, indicating all gradients have been forwarded to the next-hop datacenter, the packet is copied to the control plane *X-DC Aggregation Scheduler* to complete the X-DC aggregation.

The TCP header processing introduces complexity to the data plane. However, P4 processing time is usually in nanoseconds [51], much quicker than the seconds or minutes it takes for gradient transmission [52]. Thus, the recovery process for lost gradients in P4 switches can occur fast enough to prevent TCP timeouts and unnecessary retransmissions. As a result, the proposed TCP header processing mechanism introduces minimal impact on GDML system.

While packet loss may occur when transmitting AIP packets over the WAN, the P4 ARQ in L3DML can trigger packet re-transmission using the TCP acknowledgment (ACK) mechanism during the X-DC gradient aggregation process. The P4 switch, as the sender of the AIP packets, can re-transmit any lost AIP packets, thereby ensuring lossless gradient transmission. In addition, the loss of ACK packets can lead to TCP timeouts, which may result in unnecessary retransmissions by the sender. However, our designs primarily focus on the wired networks where the packet loss is mainly caused by the link congestion. The proposed AIP addressing mechanism allows the control plane to select less congested P4 switches for gradient aggregation, thus avoiding ACK packet loss.

D. DRL-Based Rate Synchronization Routing

To obtain the final aggregated results, the X-DC aggregation should be performed after the I-DC aggregation. However, the model update of the DT application is required to be timely, irrespective of WAN bandwidth constraints. Moreover, the computational capacities of different datacenters exhibit significant disparities. Therefore, stragglers can significantly impact the performance of the entire GDML system.

To handle the straggler problem, existing solutions typically target the DT within a datacenter network. In the context of GDML, addressing the straggler problem requires consideration of both the computational capacities of various datacenters and WAN bandwidth constraints. Moreover, solving the straggler problem involves establishing an optimal X-DC aggregation order, which is a per-hop gradient aggregation process starting from multiple datacenters, aggregating gradients at each subsequent datacenter until all participating datacenters are covered. This complexity poses a dynamic online optimization problem.

Data transmission between datacenters introduces delays, enabling slower workers to complete their current training rounds while faster datacenters transmit gradients, thereby reducing unnecessary waiting time. In this context, L3DML formulates the straggler problem as a rate synchronization routing problem and introduces a DRL-based routing approach to tackle the straggler problem.

1) Problem Formulation: Given a set of datacenters and their interconnecting bandwidth, the goal of the straggler problem is to determine routes with minimum metric (the average time cost for X-DC aggregation) among datacenters, ensuring that the gradients from faster datacenters traverse all remaining datacenters hop-by-hop in a specific order. The gradient traffic should pass through each datacenter and ultimately reach the slowest straggler. An example is shown in Fig. 7.

In L3DML, a datacenter is referred to as a **node**, and the links between datacenters are referred to as **edges**. The straggler problem is defined on a complete bi-directional graph $G = (V, E)$, where V is the node set with size v and E is the edge set. V includes a subset of **starting points** S with size s, representing faster datacenters, and a subset of **checkpoints** C_n with size c_n, which are slower datacenters that gradients from starting point n ($n \in S$) will flow through. The straggler

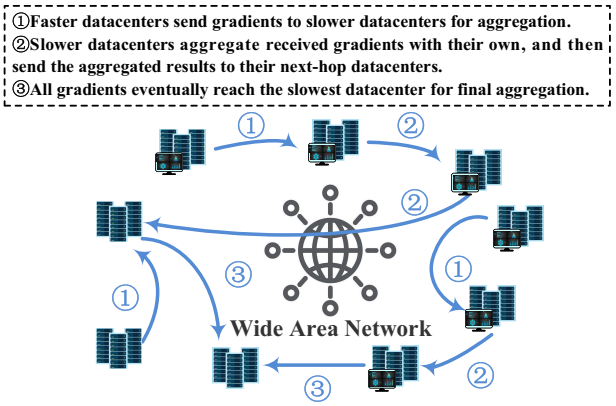


Fig. 7. An example of the rate synchronization routing in GDML scenarios.

problem considers asymmetric distances. Thus, E contains edges in both directions.

For a specific starting point n, it initiates gradient transmission at the beginning time (t_0^n). Gradients flow through a sequence of checkpoints, aggregating with the gradients at each checkpoint. Ultimately, gradients from multiple starting points converge at the straggler o. The metric of the edge from checkpoint i to checkpoint j consists of two parts defined by Eq. (3) and Eq. (4).

$$T_{i,j}^n(t_i') = \frac{G_i}{B_{i,j}(t_i')} + P_{i,j} \tag{3}$$

$$W_{i,j}^n(t_i) = \begin{cases} T_j - t_i, & t_i < t_0^n + T_j \\ 0, & t_i \geq t_0^n + T_j \end{cases} \tag{4}$$

$T_{i,j}^n(t_i')$ denotes the time cost for checkpoint i to transmit gradients to checkpoint j for the given starting point n, where t_i' is the time when checkpoint i completes its I-DC aggregation process. G_i represents the gradient size of checkpoint i in the current round of X-DC aggregation process. $B_{i,j}(t_i')$ denotes the bandwidth between checkpoint i and checkpoint j at time t_i'. $P_{i,j}$ is the propagation delay of the link between checkpoint i and checkpoint j.

$W_{i,j}^n(t_i)$ refers to the waiting time cost at checkpoint j before aggregating gradients from checkpoint i for the given starting point n, where t_i indicates the time when checkpoint i finishes its X-DC gradient transmission. T_j denotes the time taken by checkpoint j to complete its current training round and I-DC aggregation process.

Therefore, the metric of the edge from checkpoint i to checkpoint j for starting point n is $T_{i,j}^n(t_i') + W_{i,j}^n(t_i)$. To simplify computations, we assume that the bandwidth between checkpoints remains constant during gradient transmission, implying $T_{i,j}^n(t_i) = T_{i,j}^n(t_i')$. Hence, the metric of the edge from checkpoint i to checkpoint j is expressed as a time-dependent function defined by Eq. (5).

$$M_{i,j}^n(t) = T_{i,j}^n(t) + W_{i,j}^n(t) \tag{5}$$

Let $x_{i,j}$ represent a binary decision variable that is equal to 1 if the checkpoint i transmits its gradients to checkpoint j, and 0 otherwise. The objective is to minimize the average time consumption for completing X-DC aggregation across all starting points, achieving rate synchronization to prevent

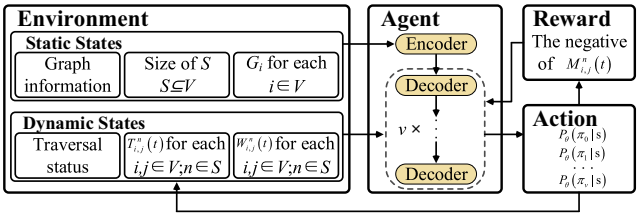


Fig. 8. Proposed DRL framework.

nodes from aggregation failing due to excessive waiting. The definition of the objective function is depicted in Eq. (6):

$$\text{minimize } \frac{1}{s} \sum_{n \in S} \sum_{i \in C_n} \sum_{j \in C_n} x_{i,j} M_{i,j}^n(t_i) \tag{6}$$

with the following constraints:

$$\exists j \in V, j \notin S : \forall i \in V, i \neq o, x_{i,j} = 1 \tag{7}$$

$$\exists j \in V : \forall i \in V, B_{i,j}(t_i) \neq 0 \tag{8}$$

$$\exists! j \in V : \forall i \in V, \sum_{i \in V} \sum_{j \in V} x_{j,i} = 0, \sum_{i \in V} \sum_{j \in V} x_{i,j} \neq 0 \tag{9}$$

$$\forall n \in S, i \in C_n, j \in C_n, x_{i,j} M_{i,j}^n(t_i) = x_{i,j}\left(t_j' - t_i'\right) \tag{10}$$

Constraint (7) ensures that all nodes are traversed in the X-DC aggregation process. Constraint (8) guarantees the availability of WAN bandwidth resources for nodes during gradient transmission. Constraint (9) ensures that gradients from all starting points can ultimately converge to the same node. Constraint (10) guarantees gradient arrival times at each checkpoint increase along the path to prevent re-touring.

2) Decision Transformer DRL Framework: Existing solutions to the straggler problem do not involve per-hop gradient processing. They only focus on selecting the optimal path for each worker to ensure gradients from all workers arrive at the aggregation point simultaneously. In contrast, L3DML differs from them since L3DML requires parallel decision-making for multiple starting points, gradually selecting the next-hop nodes. The decision made for each starting point affects the decisions of other starting points. Therefore, L3DML employs a decision transformer-based DRL approach [53]. This approach leverages the multi-head attention mechanism of the transformer model to enhance the interrelation among decisions of various starting points. Additionally, it applies DRL to maximize long-term benefits. Table I summarizes the notations to be used in L3DML.

We consider the straggler problem as a Markov decision process. The proposed decision transformer DRL framework comprises four components, as depicted in Fig. 8.

The **environment** includes both static and dynamic states. Static states consist of graph information (node identities and interconnection information), the size of S, and G_i for each $i \in V$. Dynamic states encompass the traversal status, $T_{i,j}^n(t)$ and $W_{i,j}^n(t)$ for each $i, j \in V$; $n \in S$s. In this paper, once a certain node initiates X-DC gradient transmission, it is regarded as a traversed node, thereby preventing it from being re-selected as a checkpoint. The **agent** is represented by an encoder-decoder model, denoted as θ. The agent includes one encoder and v decoders and is used to train a stochastic policy

TABLE I
KEY NOTATIONS

Notations	Definition
G_i	the gradient size to be transmitted by checkpoint i in the current round of X-DC aggregation process
$P_{i,j}$	the propagation delay of the link between checkpoint i and checkpoint j
$B_{i,j}(t)$	the bandwidth between checkpoint i and checkpoint j at time t
T_i	the time taken by checkpoint i to complete its current training round and I-DC aggregation process
t_i	the time when checkpoint i finishes its X-DC gradient transmission
$T^n_{i,j}(t)$	the time cost for checkpoint i to transmit gradients to checkpoint j for the given starting point n
$W^n_{i,j}(t)$	the waiting time cost at checkpoint j before aggregating gradients from checkpoint i for the given starting point n
$M^n_{i,j}(t)$	the metric of the edge from checkpoint i to checkpoint j for the given starting point n
$\mathcal{N}$	a fully connected neural network layer
$\mathcal{B}$	a batch normalization sub-layer
$\mathcal{F}$	a feed-forward sub-layer with one hidden layer
$\mathcal{A}$	a multi-head attention sub-layer
x_i	the identification of node i, a v-dimensional one-hot vector
y_i	the calculated metric of edges between node i and other nodes at time t_0, a v-dimensional vector
$[a, b]$	the concatenation of vectors a and b
X_i	the node ID embedding of node i, a 128-dimensional vector
Y_i	the edge metric embedding of node i, a 128-dimensional vector
$h_i^{(j)}$	the input of the j^{th} layer, also the output of the $(j-1)^{\text{th}}$ layer, a 128-dimensional vector
h_i	the aggregated embedding of node i, a 128-dimensional vector
$t^n_{k_i}$	the time when starting point n make its k^{th} decision
w^t_i	the calculated metric of the edges between node i and other nodes at time t, a v-dimensional vector
w^t	the embedding of the edge metric at time t, a 128-dimensional vector
u^t_i	the traversal status of node i at time t, which is equal to 1 if node i has not been traversed and 0 otherwise
u^t	the embedding of the traversal status at time t, a 128-dimensional vector
ε^t	the embedding of the current state
h	the embedding of the entire graph, a 128-dimensional vector
l	the previous checkpoint
$\otimes$	element-wise multiplication
$p^{n,k}_i$	The probability of starting point n selecting node i as the k^{th} checkpoint
π_k	the action of the k^{th} decision, an s-dimensional vector

$p_\theta(\pi|\varepsilon)$, which indicates the probability of taking **action** π (choosing a set of next-hop checkpoints for each starting point) based on state ε. By applying the probability chain rule, $p_\theta(\pi|\varepsilon)$ can be obtained using Eq. (11).

$$p_\theta(\pi|\varepsilon) = \prod_{k=1}^{v} p_\theta(\pi_k|\varepsilon, \pi_1, \ldots, \pi_{k-1}) \tag{11}$$

The encoder utilizes state information to create an embedding that abstracts the graph context. Decoders sequentially output each π_k based on the context embedding. After a decoder generates its action, dynamic states are updated, and relevant **rewards** are accumulated. The immediate reward is defined as the negative of $M^n_{i,j}(t)$.

3) Model Training Algorithm: In contrast to the straggler problem within a single datacenter, decision-making in the GDML scenario involves multiple starting points. The number of starting points determines the length of the action space $|\pi|$, with more starting points leading to a lower upper bound on the objective function.

Therefore, solving the straggler problem in the GDML scenario using a single model is challenging, as it may promote the use of additional starting points. To tackle this, L3DML introduces the concept of model set and employs a rollback-enabled policy gradient algorithm for model training, as outlined in Algorithm 1.

Algorithm 1 Rollback-Enabled Policy Gradient Algorithm

Parameter: Data samples D, number of epochs EP, number of samples per epoch m, maximum number of starting points z, rollback epoch threshold EP_{th}, rollback epoch counter ep;
Input: Objective function $J(\theta)$;
Output: Trained model set $\{\theta^*_1, \cdots, \theta^*_s\}$;

for n=1 to z **do**
  $\theta_n \leftarrow$ RandomValue;
  $\theta^*_n \leftarrow \theta_n$;
  $ep \leftarrow 0$;
  $|\pi_i| \leftarrow z, \forall i \in 1, \ldots, m$;
  for epoch=1 to EP **do**
    $x_i \leftarrow \widetilde{D}, \forall i \in 1, \ldots, m$;
    $\pi_i \leftarrow \theta_n(x_i), \forall i \in 1, \ldots, m$;
    $\pi^*_i \leftarrow \theta^*_n(x_i), \forall i \in 1, \ldots, m$;
    $\nabla\text{loss} \leftarrow \sum_{i=1}^{m}(J(\pi_i) - J(\pi^*_i))\nabla_{\theta_n}\log p_{\theta_n}(\pi_i)$;
    $\theta_n \leftarrow \text{Adam}(\theta_n, \nabla\text{loss})$;
    if ttest_rel$(J_{\theta_n}, J^*_{\theta_n})$ <0.05 **then**
      $\theta^*_n \leftarrow \theta_n$;
      $ep \leftarrow$ epoch;
    else
      if epoch-ep > EP_{th} **then**
        $\theta_n \leftarrow \theta^*_n$;
      end if
    end if
  end for
end for
return $\{\theta^*_1, \ldots, \theta^*_z\}$

Here θ denotes the current model, while θ^* represents the best model trained so far. We utilize random sampling $\widetilde{D}$ as instances for an epoch of training. We compare the performance of these two models using the paired sample t-test function $ttest\_rel()$. This statistical method compares the means of two groups of model parameters to analyze the differences between them. If θ significantly outperforms θ, then θ^* is updated. If θ changes insignificantly for EP_{th} epochs, θ is rolled back to θ^*.

4) Rate Synchronization Routing Algorithm: Given the practical state information of the network and the number of starting points, we propose a rate synchronization routing algorithm using the trained model set to cope with the straggler problem in GDML scenarios. The routing decision process is illustrated in Algorithm 2.

Encoder processing: The encoder processes node and graph embeddings by taking the identification vector x_i and metric vector y_i of each node as inputs. These inputs are

Algorithm 2 Rate Synchronization Routing Algorithm

Parameter: number of starting points s, trained model θ_s^*;
Input: x_i, y_i, $M_{i,j}^n(t)$, $u_i^{(t)}$;
Output: Action set $\{\pi_1, \cdots, \pi_v\}$;

//Encoder processing;
for $\forall i \in V$ **do**
  $X_i \leftarrow \mathcal{N}(x_i)$;
  $Y_i \leftarrow \mathcal{N}(y_i)$;
  $h_i^{(1)} \leftarrow \mathcal{N}([X_i, Y_i])$;
  $\hat{h}_i^{(j)} \leftarrow \mathcal{B}(h_i^{(j-1)} + \mathcal{A}(h_1^{(j-1)}, \ldots, h_v^{(j-1)}))$;
  $h_i^{(j)} \leftarrow \mathcal{B}(\hat{h}_i^{(j)} + \mathcal{F}(\hat{h}_i^{(j)}))$;
  $h_i \leftarrow h_i^{(N)}$;
end for
$h \leftarrow \frac{1}{v}\sum_{i \in V} h_i$;
//Decoder processing;
for $k = 1$ to v **do**
  for $\forall n \in S$ **do**
    $w^{t_k^n} \leftarrow \mathcal{N}(w_1^{t_k^n}, \ldots, w_v^{t_k^n})$;
    $u^{t_k^n} \leftarrow \mathcal{N}(u_1^{t_k^n}, \ldots, u_v^{t_k^n})$;
    $\varepsilon^{t_k^n} \leftarrow \left[w^{t_k^n}, u^{t_k^n}, h, h_l\right]$;
    $\left[\bar{h}_1^n, \ldots, \bar{h}_v^n\right] \leftarrow \mathcal{F}(\mathcal{A}(\varepsilon^{t_k^n}, h_1, \ldots, h_v))$;
    $\left[\bar{p}_1^{n,k}, \ldots, \bar{p}_v^{n,k}\right] \leftarrow \left[\bar{h}_1^n, \ldots, \bar{h}_v^n\right] \otimes \left[u_1^{t_k^n}, \ldots, u_v^{t_k^n}\right]$;
    $\left[p_1^{n,k}, \ldots, p_v^{n,k}\right] \leftarrow \text{softmax}(\left[\bar{p}_1^{n,k}, \ldots, \bar{p}_v^{n,k}\right])$;
    $\pi_k(n) \leftarrow \text{argmax}(\left[p_1^{n,k}, \ldots, p_v^{n,k}\right])$;
  end for
end for
return $\{\pi_1, \ldots, \pi_v\}$

transformed into node ID embedding X_i and the edge metric embedding Y_i through a fully connected neural network. The resultant embeddings are then concatenated to generate hidden embeddings $h_1^{(1)}, \ldots, h_v^{(1)}$. Finally, the encoder computes the aggregated embedding h_i for node i using N attention layers, each including a multi-head attention sub-layer and a node-wise fully connected feed-forward sub-layer with one hidden layer and activation function.

Decoder processing: Due to asynchronous decision-making at each starting point, it is difficult for the decoder to calculate the edge metric. Hence, we assume that for any starting point n, decision-making for the $(i+1)^{\text{th}}$ hop only occurs when the i^{th} checkpoint has finished receiving gradients from the previous checkpoint, i.e., $t_{i+1}^n = t_i, \forall i \in V$. As a result, the time horizon can be discretized by the time t_i. To make decisions for each starting point in parallel, the proposed decoder is reused in v steps. At each step, embeddings w^t and u^t are generated using the calculated edge metric and traversal status of each node. These embeddings, along with h and the previous checkpoint feature h_l, are concatenated to represent the embedding of the current state. The attention mechanism is then applied to context embeddings and aggregated node embeddings. After masking traversed nodes, a **softmax** layer is used to obtain the probability of selecting an untraversed node as the next checkpoint. Finally, the next checkpoint for each starting point is determined using the **argmax** function, and the traversal status and decision time for the next step are updated. Following this, the next decoding step is performed until all checkpoints for each starting point are determined.

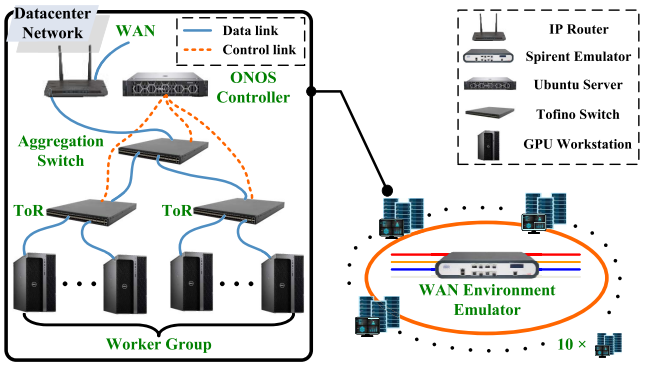


Fig. 9. Topology of the prototype system.

V. Implementation and Evaluation

A. Prototype Implementation

Our prototype system comprises 10 datacenter networks interconnected via a WAN environment emulator. Each datacenter network includes 2 ToR switches, 1 aggregation switch, and 1 controller. Within each datacenter network, 2-4 workers are randomly assigned, which are connected to different ToR switches. ToR switches link to the aggregation switch, enabling in-network gradient aggregation. For control plane management, we utilize the Open Network Operation System (ONOS) [54] as the controller, with the proposed control plane functions deployed as ONOS APPs. The topology of our prototype system is depicted in Fig. 9.

Hardware parameters: The P4 programmable switch utilized in our prototype is the Barefoot Networks Wedge BF-48X6Z, operating on the Open Network Linux OS (ONL-master) system. It is equipped with an Intel Xeon CPU 1527 @2.20GHz and 16GB of memory. Each worker is a Dell Workstation running Ubuntu 22.04, equipped with an Intel Xeon CPU 4210R @2.4GHz, 20 Cores, 128GB of memory, and two RTX A4000 GPUs, each with 16GB of graphics memory. The controller is a DELL EMC server running Ubuntu 18.04, equipped with an Intel Xeon Silver CPU 4214 @2.20GHz, 12 Cores, and 64GB of memory. To emulate the WAN environment, we employed the Spirent Attero-X and several gigabit IP routers.

B. Data Plane Performance Evaluation

To evaluate the data plane performance of L3DML, we focus on the following metrics:

Maximum throughput for forwarding AIP packets of various sizes: The processing time cost per packet directly affects network throughput. Considering the increased complexity of processing logic for AIP packets compared with traditional IP packets, evaluating line-rate processing throughput (non-blocking network-layer processing) is crucial.

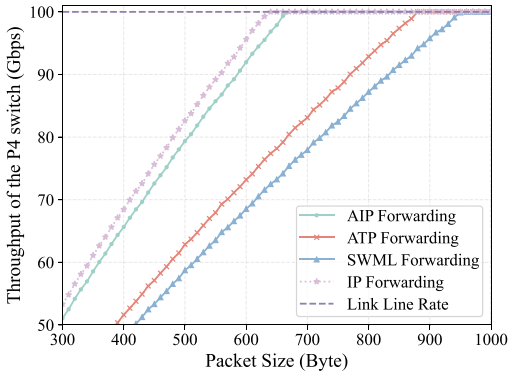


Fig. 10. Throughput of the P4 switch vs. packet sizes.

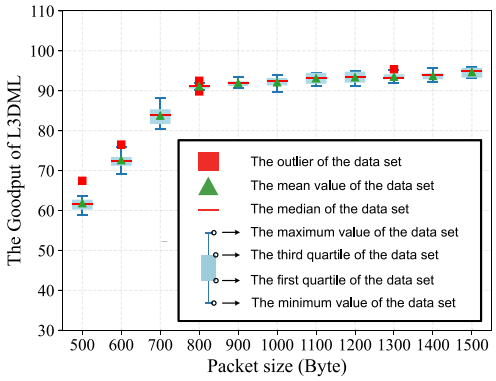


Fig. 11. Goodput of L3DML.

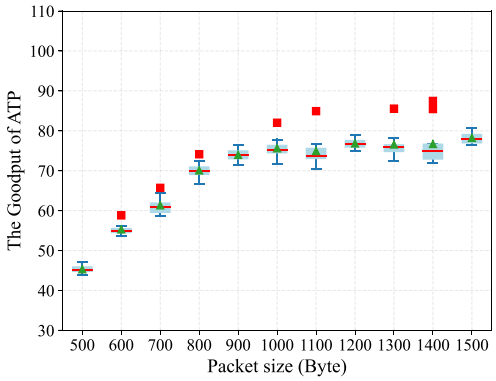


Fig. 12. Goodput of ATP.

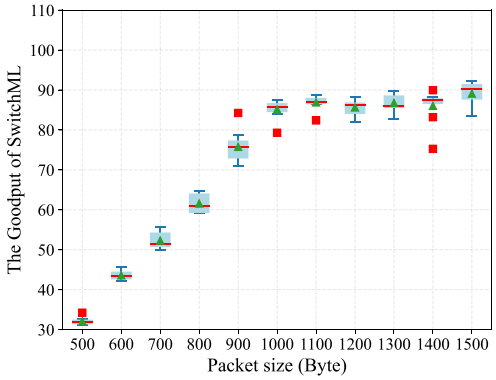


Fig. 13. Goodput of SwitchML.

Aggregation goodput of the P4 switch: The processing logic of the P4 pipeline involved in gradient aggregation is complex, which directly impacts the packet processing latency at the aggregation point. Therefore, it is essential to evaluate the goodput (measured as the maximum size of gradients aggregated within a second) of the P4 switch.

Processing latency of different modules in L3DML: To provide a comprehensive overview of the processing latency incurred within P4 switches by various processing operations, we measured the processing latency of different modules in L3DML while handling AIP packets of varying sizes.

To conduct the test, we introduce AIP traffic into the ToR switch of our prototype system at a rate of 100 Gbps. The packet size is increased by 10 bytes from 300 bytes to 1,000 bytes. We measure the average throughput of the P4 switch for forwarding AIP packets of varying sizes. For comparative analysis, we also perform similar tests using ATP [25], SwitchML [5], and IP packets. Fig. 10 shows the results.

In Fig. 10, AIP forwarding demonstrates superior performance over ATP and SwitchML. The threshold packet size for our design to achieve line-rate forwarding is 662 bytes, whereas it is 879 and 947 bytes for ATP and SwitchML, respectively. This is because our design utilizes simple match-action operations for processing AIP packets, avoiding complex packet recirculation. Conversely, ATP and SwitchML necessitate complex gradient aggregation processing before packet forwarding. The results also indicate that AIP forwarding performance is slightly lower than that of legacy IP forwarding, with a threshold packet size of 638 bytes. Nonetheless, AIP forwarding can rapidly achieve line-rate performance as the packet size increases. In conclusion, AIP packet forwarding exhibits better performance than existing solutions and can be implemented at the network layer with a slightly higher cost than IP forwarding.

To evaluate the aggregation goodput of the P4 switch, we inject 100 Gbps AIP traffic, including TCP header and gradients, into the aggregation switch. Then, we gradually increase the packet size by 100 bytes, starting from 500 bytes up to the MTU of 1500 bytes. We monitor the goodput of the P4 switch using the Tofino log. The goodput measurement is conducted ten times for each packet size and presented using a box plot. For comparative analysis, we also conduct similar tests using the representative solutions SwitchML [5] and ATP [25] as the benchmark. The results are depicted in Fig. 11, Fig. 12, and Fig. 13. We also measure the processing latency of different modules in L3DML while handling AIP packets of varying sizes, as shown in Fig. 14.

Fig. 11, 12, and 13 demonstrate the comparative performance of L3DML, ATP, and SwitchML regarding the goodput. L3DML achieves the highest average goodput of 94.65 Gbps, surpassing ATP (78.23 Gbps) and SwitchML (89.1 Gbps). This is because the register utilization of ATP experiences collisions, requiring partial gradients to be sent to the PS for aggregation, thus reducing goodput. SwitchML relies on a dedicated protocol for lossless gradient aggregation, which introduces additional processing latency. In contrast, L3DML does not depend on the PS or a dedicated protocol, and uses *jobID* to partition register usage, thereby avoiding

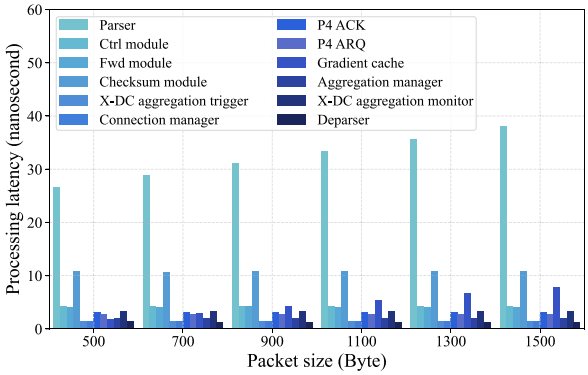


Fig. 14. Processing latency of different modules in L3DML.

collisions. The results also show that L3DML outperforms ATP and SwitchML when using small packets with sizes below 800 Bytes. Small gradient packets result in a higher number of packets, increasing the likelihood of collisions in ATP register utilization. SwitchML uses a shadow copy mechanism to prevent redundant aggregation and introduces packet recirculation. This further increases memory usage on the switch when processing smaller packets.

As illustrated in Fig. 14, the parser and checksum modules account for the majority of the latency in AIP packet processing. This is primarily due to the necessity of the P4 parser to extract all information contained within the AIP packets, which significantly consumes system resources. Additionally, the checksum module necessitates state maintenance across packets for aggregation verification, thereby introducing further latency. In contrast, the latency contributed by the other modules is relatively lower, enabling L3DML to achieve line-rate packet processing. Furthermore, Fig. 14 indicates that as the size of AIP packets increases, the processing latency of both the parser and gradient cache also rises. This increase is a natural consequence of the data plane to extract and process a larger volume of gradients within the payload. Nevertheless, L3DML is capable of achieving line-rate gradient aggregation without exhibiting scalability issues.

C. Rate Synchronization Routing Analysis

The performance of X-DC aggregation is directly affected by the number of datacenters and starting points. To investigate the impact of the number of datacenters and starting points on our proposed rate synchronization routing mechanism, we measured the average time cost of the paths determined by each starting point with varied numbers of datacenters (n) and starting points (s).

We employ an Alibaba PAI (Platform for Artificial Intelligence) dataset [55] to train the model set of the proposed rate synchronization routing mechanism. This dataset offers comprehensive records for running state-of-the-art ML algorithms. Afterward, we integrate the trained model set into the ONOS routing APP and simulate the proposed rate synchronization routing mechanism using an ONOS cluster. The average time cost of the paths determined by each starting point is measured in Fig. 15

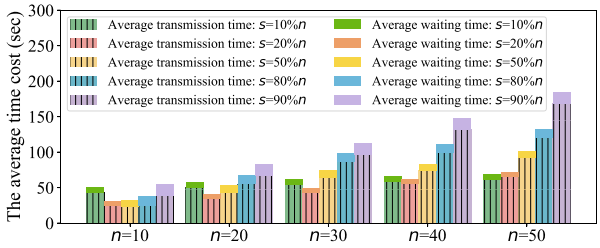


Fig. 15. Average time cost of the paths determined by each starting point.

In Fig. 15, we can see that increasing the number of starting points ($s = 20\%n$) can effectively reduce the average time cost if $n < 40$. This is mainly because multiple starting points can perform X-DC aggregation in parallel, thereby improving the speed. However, parallel X-DC aggregation may degrade if $s > 20\%n$, especially for $n \geq 40$, due to bandwidth contention. Moreover, a higher number of s can lead to a longer average waiting time since faster datacenters are selected as starting points, thus slower datacenters become the next hop in rate synchronization routing. Fig. 15 also indicates that, as the number of n increases, the average transmission time increases, while the average waiting time remains nearly constant. This occurs because the waiting time primarily depends on the computational capabilities of the workers, whereas a larger n increases the average hop count of the path determined by each starting point, resulting in a greater average transmission time.

D. Performance Evaluation for GDML

We mainly focus on the following metrics to evaluate the performance of the GDML system:

The reference loss of the global model: Gradient packet loss can result in incomplete gradient aggregation, which subsequently impacts the accuracy of the global model. To evaluate the effect of L3DML on model accuracy in GDML, we employ the average loss of model parameters (referred to as reference loss) at various training rounds as the metric.

The average size of transmitted gradients: The size of the gradients transmitted during X-DC aggregation directly impacts the training speed of the GDML system. Hence, we utilize the average size of transmitted gradients over the WAN as the metric to assess the efficiency of gradient aggregation.

The idle waiting time of GDML: The straggler problem introduces unnecessary waiting when aggregation gradients among datacenters. Hence, we compare the idle waiting time with existing GDML systems to demonstrate the rate synchronization routing performance of L3DML.

The training speedup for GDML: We assess the training speedup performance of L3DML and compare it with existing GDML systems as well as baseline approaches, to demonstrate an overall performance enhancement of GDML.

In our experiment, we utilize the so-vits-svc 4.0 vocal model [56] and VGGNet image model [57] on workers in our prototype system. Then, GDML is performed among 10 participating datacenters with a batch size of 16 and a learning rate of 0.0002. The raw data set comprises 1 hour of voice samples and 10GB of image samples. For benchmark selection, we compare against state-of-the-art GDML systems,

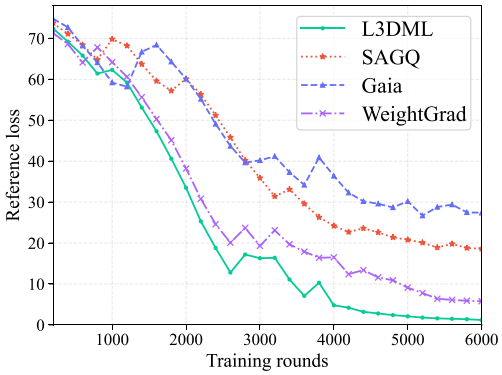


Fig. 16. Reference loss of vocal model.

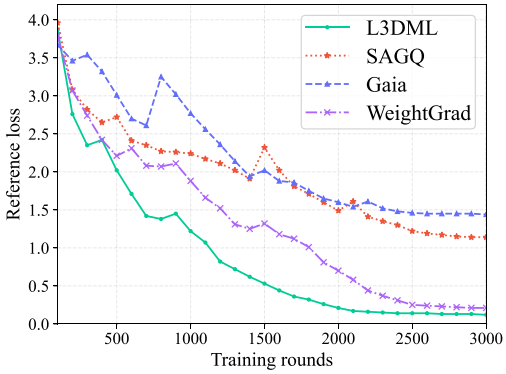


Fig. 17. Reference loss of image model.

including Gaia [3], WeightGrad [12], and SAGQ [11]. We measure the reference loss of the vocal model every 200 rounds (100 rounds for the image model). The average size of transmitted gradients is obtained from Spirent emulators.

Fig. 16 and Fig. 17 illustrate that L3DML achieves the fastest convergence in reference loss for training both vocal and image models when compared with SAGQ, WeightGrad, and Gaia. L3DML also obtains the smallest convergence loss of 1.2 and 0.13 for the vocal and image models respectively. In contrast, SAGQ, WeightGrad, and Gaia achieve convergence losses of 18.6, 5.8, and 27.4 for the vocal model and 1.14, 0.21, and 1.44 for the image model. This is because L3DML aggregates each round of gradients as opposed to discarding insignificant gradient packets. Moreover, L3DML incorporates TCP processing logic to support lossless aggregation. In contrast, SAGQ primarily focuses on minimizing gradient transmission time cost, resulting in discarding a larger quantity of gradient packets, while Gaia and WeightGrad introduce dropout thresholds, excluding numerous packets with small gradients, thereby affecting model accuracy. In conclusion, L3DML achieves superior performance in model accuracy.

Fig. 18 presents the average size of transmitted gradients over the WAN during the entire GDML process. It can be observed that L3DML transmits the fewest gradients (389.65GB and 416.26GB for the vocal and image models respectively) compared to SAGQ, WeightGrad, and Gaia. This reduction is mainly due to the in-network aggregation

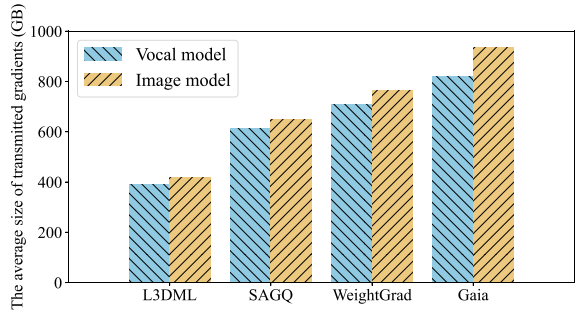


Fig. 18. Average size of transmitted gradients over the WAN.

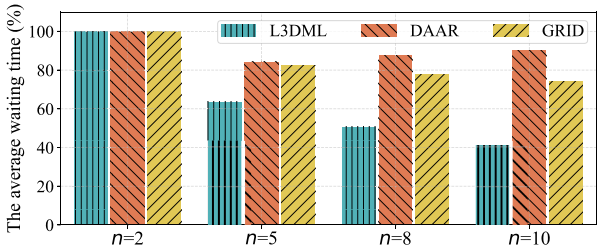


Fig. 19. Average idle waiting time compared to the baseline system.

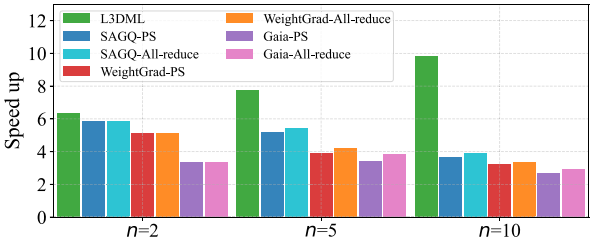


Fig. 20. Training speedup for GDML.

of L3DML, where gradients within datacenters are aggregated and further reduced among datacenters, significantly decreasing the transmitted gradient size. In contrast, SAGQ adjusts the dropout of gradient packets dynamically based on WAN bandwidth, leading to increased dropout rates under greater participating datacenters, while WeightGrad and Gaia prioritize model accuracy by discarding packets with small gradient values, resulting in limited data reduction efficiency.

To assess the rate synchronization routing performance of L3DML, we establish a baseline GDML system utilizing PyTorch with all-reduce architecture. Then, we perform GDML in our prototype with various numbers of participating datacenters to simultaneously train the vocal and image model. Similar tests are performed using the rate synchronization routing approaches of DAAR [48] and GRID [7]. The average idle waiting time compared to the baseline system is depicted in Fig. 19.

Fig. 19 demonstrates that L3DML consistently outperforms both DAAR and GRID, irrespective of the value of n. This is primarily because the rate synchronization routing of L3DML considers both network transmission time and computational delays within datacenters, whereas DAAR focuses solely on network transmission as the main factor and employs a greedy algorithm for routing decisions, resulting in limited

performance in reducing idle waiting time. Additionally, GRID assumes a single starting point for routing decisions, leading to scalability issues when n is large. In contrast, the model set of L3DML allows for parallel routing decisions from multiple starting points, thereby offering superior scalability.

To evaluate the training speed gain of L3DML, we establish a baseline GDML system utilizing PyTorch with PS-worker architecture. Then, we perform GDML in our prototype with various numbers of participating datacenters to simultaneously train the vocal and image model. We also conduct similar tests using SAGQ, WeightGrad, and Gaia with two X-DC aggregation architectures (PS-worker and all-reduce). The training speedup performance is illustrated in Fig. 20.

In Fig. 20, the results demonstrate that L3DML achieves the highest training speed gain, with a speedup of up to $6.34\times$ compared to the baseline when $n = 2$, and a speedup of $9.81\times$ when $n = 10$. In contrast, SAGQ, WeightGrad, and Gaia achieve speedups ranging from $3.63 \times -5.86\times$, $3.21\times-5.14\times$, and $2.68\times-3.41\times$, respectively, for different values of n. L3DML's superior gradient aggregation performance and effective rate synchronization routing contribute to its advantages. Conversely, SAGQ and WeightGrad overlook the negative impact of stragglers. Therefore, as the number of participating data centers increases, the performance gains of these two methods are limited, despite SAGQ's ability to dynamically adjust the gradient dropout based on WAN bandwidth. On the other hand, Gaia incorporates the Approximate Synchronous Parallel (ASP) aggregation mechanism, which filters out gradients from slower datacenters using a time threshold. This mechanism mitigates the impact of stragglers when a small number of datacenters are involved ($n = 5$). However, when the number of participating datacenters is large ($n = 10$), the ASP mechanism waits for stragglers to ensure model accuracy, resulting in additional waiting time. Furthermore, the results indicate that utilizing the all-reduce architecture for X-DC aggregation can further enhance training speed, as it maximizes the utilization of bandwidth among data centers. In conclusion, L3DML is an effective solution to enhance the performance and scalability of the GDML system.

VI. Conclusion

In this paper, we introduce L3DML, a novel network-layer solution that utilizes P4-based SDN to facilitate GDML. L3DML incorporates an innovative AIP addressing scheme and implements it using the P4 language, enabling location-specific and lossless in-network gradient aggregation. Additionally, we develop a transformer DRL-based rate synchronization routing mechanism to address the straggler problem. To validate our approach, we implement a prototype system. The experimental results demonstrate that L3DML achieves line-rate packet delivery and surpasses existing solutions in terms of goodput, model accuracy, and training speed gain for large-scale GDML.

In our future research, we will prioritize exploring the deployment of model parallel DT applications on L3DML and further evaluating the applicability of L3DML.

Acknowledgment

The authors are grateful to the reviewers and the associate editor for their constructive and insightful comments that helped us to improve the manuscript significantly.

References

[1] M. Abadi et al., "TensorFlow: A system for large-scale machine learning," in *Proc. OSDI*, vol. 16, Savannah, GA, USA, 2016, pp. 265–283.
[2] I. Cano, M. Weimer, D. Mahajan, C. Curino, and G. M. Fumarola, "Towards geo-distributed machine learning," 2016, *arXiv:1603.09035*.
[3] K. Hsieh et al., "Gaia: Geo-distributed machine learning approaching LAN speeds," in *Proc. NSDI*, 2017, pp. 629–647.
[4] Q. Zhang, L. T. Yang, Z. Chen, and P. Li, "A survey on deep learning for big data," *Inf. Fusion*, vol. 42, pp. 146–157, Jul. 2018.
[5] A. Sapio et al., "Scaling distributed machine learning with in-network aggregation," 2019, *arXiv:1903.06701*.
[6] J. Chen, X. Pan, R. Monga, S. Bengio, and R. Jozefowicz, "Revisiting distributed synchronous SGD," 2016, *arXiv:1604.00981*.
[7] J. Fang, G. Zhao, H. Xu, C. Wu, and Z. Yu, "GRID: Gradient routing with in-network aggregation for distributed training," *IEEE/ACM Trans. Netw.*, vol. 31, no. 5, pp. 2267–2280, Oct. 2023.
[8] P. Watcharapichat, V. L. Morales, R. C. Fernandez, and P. Pietzuch, "Ako: Decentralised deep learning with partial gradient exchange," in *Proc. 7th ACM Symp. Cloud Comput.*, 2016, pp. 84–97.
[9] B. Recht, C. Re, S. Wright, and F. Niu, "Hogwild! A lock-free approach to parallelizing stochastic gradient descent," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 24, 2011, pp. 693–701.
[10] Q. Ho et al., "More effective distributed ML via a stale synchronous parallel parameter server," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 26, 2013, pp. 1223–1231.
[11] C. Fan, X. Zhang, Y. Zhao, Y. Liu, and S. Yu, "Self-adaptive gradient Quantization for geo-distributed machine learning over heterogeneous and dynamic networks," *IEEE Trans. Cloud Comput.*, vol. 11, no. 4, pp. 3483–3496, Oct.–Dec. 2023.
[12] S. N. Akter and M. A. Adnan, "WeightGrad: Geo-distributed data analysis using quantization for faster convergence and better accuracy," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Disc. Data Min.*, 2020, pp. 546–556.
[13] B. Heintz, A. Chandra, and R. K. Sitaraman, "Optimizing grouped aggregation in geo-distributed streaming analytics," in *Proc. 24th Int. Symp. High-Perform. Parallel Distrib. Comput.*, 2015, pp. 133–144.
[14] C.-C. Hung, L. Golubchik, and M. Yu, "Scheduling jobs across geo-distributed datacenters," in *Proc. 6th ACM Symp. Cloud Comput.*, 2015, pp. 111–124.
[15] K. Kloudas, M. Mamede, N. Preguiça, and R. Rodrigues, "Pixida: Optimizing data parallel jobs in wide-area data analytics," *Proc. VLDB Endow.*, vol. 9, no. 2, pp. 72–83, 2015.
[16] Q. Pu et al., "Low latency geo-distributed data analytics," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 45, no. 4, pp. 421–434, 2015.
[17] A. Rabkin, M. Arye, S. Sen, V. S. Pai, and M. J. Freedman, "Aggregation and degradation in {*JetStream*}: Streaming analytics in the wide area," in *Proc. 11th USENIX Symp. Netw. Syst. Design Implement. (NSDI)*, 2014, pp. 275–288.
[18] R. Viswanathan, G. Ananthanarayanan, and A. Akella, "CLARINET: WAN-aware optimization for analytics queries," in *Proc. OSDI*, vol. 16, 2016, pp. 435–450.
[19] A. Vulimiri et al., "WANalytics: Geo-distributed analytics for a data intensive world," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2015, pp. 1087–1092.
[20] A. Vulimiri, C. Curino, P. B. Godfrey, T. Jungblut, J. Padhye, and G. Varghese, "Global analytics in the face of bandwidth and regulatory constraints," in *Proc. 12th {USENIX} Symp. Netw. Syst. Design Implement. ({NSDI})*, 2015, pp. 323–336.
[21] L. Chen, S. Liu, B. Li, and B. Li, "Scheduling jobs across geo-distributed datacenters with max-min fairness," *IEEE Trans. Netw. Sci. Eng.*, vol. 6, no. 3, pp. 488–500, Jul.–Sep. 2019.
[22] L. Liu, H. Yu, G. Sun, L. Luo, Q. Jin, and S. Luo, "Job scheduling for distributed machine learning in optical WAN," *Future Gener. Comput. Syst.*, vol. 112, pp. 549–560, Nov. 2020.
[23] H. Zhou et al., "TSEngine: Enable efficient communication overlay in distributed machine learning in WANs," *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 4, pp. 4846–4859, Dec. 2021.

[24] D. Sanvito, G. Siracusano, and R. Bifulco, "Can the network be the AI accelerator?" in *Proc. Morning Workshop Netw. Comput.*, 2018, pp. 20–25.
[25] C. Lao et al., "ATP: In-network aggregation for multi-tenant learning," in *Proc. NSDI*, vol. 21, 2021, pp. 741–761.
[26] R. L. Graham et al., "Scalable hierarchical aggregation protocol (SHArP): A hardware architecture for efficient data reduction," in *Proc. 1st Int. Workshop Commun. Optim. HPC (COMHPC)*, 2016, pp. 1–10.
[27] Y. Li, I.-J. Liu, Y. Yuan, D. Chen, A. Schwing, and J. Huang, "Accelerating distributed reinforcement learning with in-switch computing," in *Proc. 46th Int. Symp. Comput. Archit.*, 2019, pp. 279–291.
[28] A. Sapio, I. Abdelaziz, A. Aldilaijan, M. Canini, and P. Kalnis, "In-network computation is a dumb idea whose time has come," in *Proc. 16th ACM Workshop Hot Topics Netw.*, 2017, pp. 150–156.
[29] S. Liu et al., "NetReduce: RDMA-compatible in-network reduction for distributed DNN training acceleration," 2020, *arXiv:2009.09736*.
[30] Z. Li et al., "A2TP: Aggregator-aware in-network aggregation for multi-tenant learning," in *Proc. 18th Eur. Conf. Comput. Syst.*, 2023, pp. 639–653.
[31] H. Lee, J. Lee, H. Kim, and S. Pack, "Straggler-aware in-network aggregation for accelerating distributed deep learning," *IEEE Trans. Services Comput.*, vol. 16, no. 6, pp. 4198–4204, Nov./Dec. 2023.
[32] F. Hauser et al., "A survey on data plane programming with P4: Fundamentals, advances, and applied research," *J. Netw. Comput. Appl.*, vol. 212, Mar. 2023, Art. no. 103561.
[33] W. Luping, W. Wei, and L. Bo, "CMFL: Mitigating communication overhead for federated learning," in *Proc. IEEE 39th Int. Conf. Distrib. Comput. Syst. (ICDCS)*, 2019, pp. 954–964.
[34] J. Yuan, M. Xu, X. Ma, A. Zhou, X. Liu, and S. Wang, "Hierarchical federated learning through LAN-WAN orchestration," 2020, *arXiv:2010.11612*.
[35] C. Du, J. Xiao, and W. Guo, "Bandwidth constrained client selection and scheduling for federated learning over SD-WAN," *IET Commun.*, vol. 16, no. 2, pp. 187–194, 2022.
[36] J. Zhu, S. Li, and Y. You, "Sky computing: Accelerating geo-distributed computing in federated learning," 2022, *arXiv:2202.11836*.
[37] J. Bian, S. Ren, and J. Xu, "CAFE: Carbon-aware federated learning in geographically distributed data Centers," 2023, *arXiv:2311.03615*.
[38] Y. Low, J. Gonzalez, A. Kyrola, D. Bickson, C. Guestrin, and J. M. Hellerstein, "Distributed GraphLab: A framework for machine learning in the cloud," 2012, *arXiv:1204.6078*.
[39] M. Li, D. G. Andersen, A. J. Smola, and K. Yu, "Communication efficient distributed machine learning with the parameter server," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 27, 2014, pp. 1–9.
[40] Y. Bao, Y. Peng, C. Wu, and Z. Li, "Online job scheduling in distributed machine learning clusters," in *Proc. IEEE Conf. Comput. Commun.*, 2018, pp. 495–503.
[41] M. Yu, J. Liu, C. Wu, B. Ji, and E. S. Bentley, "Toward efficient online scheduling for distributed machine learning systems," *IEEE Trans. Netw. Sci. Eng.*, vol. 9, no. 4, pp. 1951–1969, Jul./Aug. 2022.
[42] M. M. Amiri and D. Gündüz, "Computation scheduling for distributed machine learning with straggling workers," *IEEE Trans. Signal Process.*, vol. 67, no. 24, pp. 6270–6284, Dec. 2019.
[43] H. Wang, Z. Liu, and H. Shen, "Machine learning feature based job scheduling for distributed machine learning clusters," *IEEE/ACM Trans. Netw.*, vol. 31, no. 1, pp. 58–73, Feb. 2023.
[44] P. Costa, A. Donnelly, A. Rowstron, and G. O'Shea, "Camdoop: Exploiting in-network aggregation for big data applications," in *Proc. 9th USENIX Symp. Netw. Syst. Design Implement. (NSDI)*, 2012, pp. 29–42.
[45] L. Mai et al., "NetAgg: Using middleboxes for application-specific on-path aggregation in data centres," in *Proc. 10th ACM Int. Conf. Emerg. Netw. Exp. Technol.*, 2014, pp. 249–262.
[46] P. Patarasuk and X. Yuan, "Bandwidth optimal all-reduce algorithms for clusters of workstations," *J. Parallel Distrib. Comput.*, vol. 69, no. 2, pp. 117–124, 2009.
[47] Z. Chen, X. Long, L. Chen, Y. Wu, J. Wu, and S. Liu, "Intra-cluster aggregation aware routing for distributed training in wireless sensor networks," *Concurr. Comput. Pract. Exp.*, vol. 35, no. 17, 2021, Art. no. e6795.
[48] Z. Chen, X. Long, Y. Wu, L. Chen, J. Wu, and S. Liu, "Data aggregation aware routing for distributed training," in *Proc. 21st Int. Conf. Parallel Distrib. Comput., Appl. Technol.*, 2021, pp. 241–250.
[49] M. Li et al., "Scaling distributed machine learning with the parameter server," in *Proc. 11th {USENIX} Symp. Oper. Syst. Design Implement. ({OSDI})*, 2014, pp. 583–598.
[50] J. Gao et al., "Lyra: A cross-platform language and compiler for data plane programming on heterogeneous ASICs," in *Proc. Annu. Conf. ACM Special Interest Group Data Commun. Appl., Technol., Archit., Protocols Comput. Commun.*, 2020, pp. 435–450.
[51] J. Lim, "How much does it burn? Profiling the energy model of a Tofino switch," Ph.D. dissertation, ETH Zürich, Zürich, Switzerland, 2023. [Online]. Available: http://www.poker-edge.com/stats.php
[52] E. A. Van Dis, J. Bollen, W. Zuidema, R. van Rooij, and C. L. Bockting, "ChatGPT: Five priorities for research," *Nature*, vol. 614, no. 7947, pp. 224–226, 2023.
[53] W. Kool, H. Van Hoof, and M. Welling, "Attention, learn to solve routing problems!" 2018, *arXiv:1803.08475*.
[54] P. Berde et al., "ONOS: Towards an open, distributed SDN OS," in *Proc. 3rd Workshop Hot Topics Softw. Defined Netw.*, 2014, pp. 1–6.
[55] "Alibaba PAI (platform for artificial intelligence." Accessed: Mar. 20, 2024. [Online]. Available: https://github.com/alibaba/clusterdata/tree/master/cluster-trace-gpu-v2020
[56] "So-vits-svc 4.0." Accessed: Dec. 30, 2023. [Online]. Available: https://github.com/svc-develop-team/so-vits-svc
[57] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," 2014, *arXiv:1409.1556*.

Xindi Hou received the M.S. degree in information and communication engineering from Northeast Electric Power University, Jilin, China, in 2018. He is currently pursuing the Ph.D. degree with the National Engineering Research Center for Advanced Network Technologies, Beijing Jiaotong University. His research interests include clean-slate networks, future network addressing and routing, compute-aware networks, and programmable data plane.

Shuai Gao (Member, IEEE) received the B.S., M.S., and Ph.D. degrees in communication and information systems from Beijing Jiaotong University in 2001, 2004, and 2010, respectively, where he is currently a Full Professor with the School of Electronic and Information Engineering. From 2015 to 2016, he was a Visiting Scholar with the University of Arizona. His research interests are in the areas of Internet architecture, sensor networks, and mobile Internet.

Ningchun Liu received the B.E. degree from Dalian Jiaotong University, Dalian, China, in 2016, and the Ph.D. degree in communication and information systems from Beijing Jiaotong University, Beijing, China, in 2023. He is currently a Assistant Research Fellow with the Research Institute of Intelligent Networks, Zhejiang Lab. His research interests include information-centric networks, software-defined networks, programmable data plane, and network security.

Fangtao Yao is a Postgraduate with the National Engineering Research Center for Advanced Network Technologies, Beijing Jiaotong University. His current research interests include compute-aware networks, clean-slate routing, and P4 programmable data plane.

Bo Lei is the Deputy Director with the Network Technology Research Department, Research Institute of China Telecom, and the Professor Level Senior Engineer. He is currently works as a Visiting Research Fellow with the Computer Network Information Center, Chinese Academy of Sciences, the Part-Time Professor with the Beijing University of Posts and Telecommunications, the Vice-Chairman of CCSA TC617 "Edge Computing Industry Development and Standards Promotion Committee", and the Deputy Leader of Computing Power Network Special Work Group in CCSA TC614 "Network 5.0 Technical Standards Promotion Committee". In this paper, he is responsible for guiding the research content and implementation methods of the system. His research interests include cloud-network convergence, computing power network, and future network technology.

Hongke Zhang (Fellow, IEEE) received the M.S. and Ph.D. degrees in electrical and communication systems from the University of Electronic Science and Technology of China, Chengdu, China, in 1988 and 1992, respectively. He is currently a Professor with the School of Electronic and Information Engineering, and the Director of the National Engineering Research Center for Advanced Network Technologies, Beijing Jiaotong University, Beijing, China. He has authored more than ten books and is the holder of more than 70 patents. His research has resulted in many papers, books, patents, systems, and equipment in the areas of communications and computer networks.

Sajal K. Das (Fellow, IEEE) received the Ph.D. degree in computer science from the University of Central Florida, Orlando, FL, USA, in 1988. He is currently the Curators' Distinguished Professor of computer science and the Daniel St. Clair Endowed Chair with the Missouri University of Science and Technology, Rolla, MO, USA. His extensive publications appeared in high-quality journals and refereed conference proceedings. He holds five U.S. patents and coauthored four books. His h-index is 102 with more than 43,500 citations. His research interests include UAVs, wireless sensor networks, mobile and pervasive computing, cyber-physical systems and IoT, smart environments, cyber security, machine learning, and data science. He is the recipient of 14 best paper awards, the IEEE Computer Society Technical Achievement Award for pioneering contributions to sensor networks, and the University of Missouri System President's Award for Sustained Career Excellence. He is the Founding Editor-in-Chief of Elsevier's *Pervasive and Mobile Computing*, and an Associate Editor of the IEEE TRANSACTIONS ON DEPENDABLE AND SECURE COMPUTING, the IEEE TRANSACTIONS ON SUSTAINABLE COMPUTING, the IEEE/ACM TRANSACTIONS ON NETWORKING, and *ACM Transactions on Sensor Networks*.